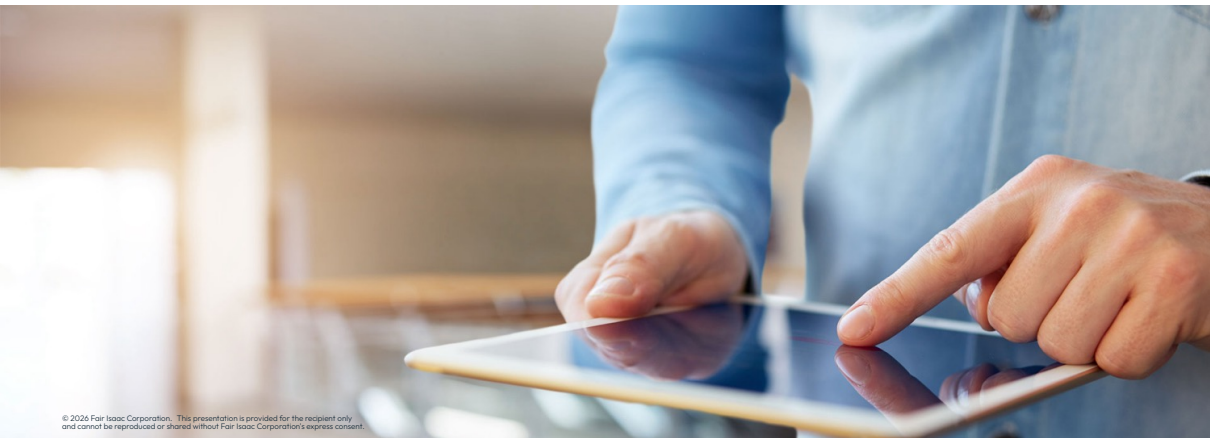


Optimizing with multiple objectives





Optimizing with multiple objectives

Optimizing for different objectives sequentially

- The `problem.setObjective()` method allows users to add several **linear** objectives for solving a problem for different objectives **sequentially**:
 - Multiple calls to `p.setObjective()` are allowed
 - The user **must define the `objidx` argument** (with consecutive integers starting with 0) to indicate the multi-objective context and the sequence of objectives to consider
 - The model is run for each objective sequentially, thus **runs are independent of each other**
 - The **sense** of the the first objective (`objidx=0`) defines the default optimization sense for all objectives:
 - To **reverse the optimization sense** for secondary objectives, set the `weight` attribute to `-1`

Optimizing for different objectives sequentially

- The `problem.setObjective()` method allows users to add several **linear** objectives for solving a problem for different objectives **sequentially**:
 - Multiple calls to `p.setObjective()` are allowed
 - The user **must define the `objidx` argument** (with consecutive integers starting with 0) to indicate the multi-objective context and the sequence of objectives to consider
 - The model is run for each objective sequentially, thus **runs are independent of each other**
 - The sense of the the first objective (`objidx=0`) defines the default optimization sense for all objectives:
 - To **reverse the optimization sense** for secondary objectives, set the weight attribute to `-1`

```
p.setObjective(x1, objidx=0)           # minimize first objective
p.setObjective(x2, objidx=1, weight=-1) # maximize second objective
p.setObjective(...)                   # other objectives

p.optimize()
```

- The Optimizer will **print the logs for each sequential run** and, in the end, a summary of the objective values found for each run:
 - This can be useful to assess the maximum possible value for each objective

Optimizing with multiple objectives

- The `problem.addObjective()` method allows users to add one or more **linear** objectives for solving **multi-objective optimization** problems:
 - Use `p.addObjective()`, possibly after an initial call to `p.setObjective()`, to create **additional objectives** (existing objectives will remain in the same problem):

```
p.addObjective(obj1,obj2,...,priority=None,weight=None,abstol=None,reltol=None)
```

Optimizing with multiple objectives

- The `problem.addObjective()` method allows users to add one or more **linear** objectives for solving **multi-objective optimization** problems:
 - Use `p.addObjective()`, possibly after an initial call to `p.setObjective()`, to create **additional objectives** (existing objectives will remain in the same problem):

```
p.addObjective(obj1,obj2,...,priority=None,weight=None,abstol=None,reltol=None)
```

- With at least one objective expression and a set of *optional* arguments:
 - `obj1, obj2, ...`: expression(s) for the objective(s) to be added to the problem
 - *priority*: priority for the new objective(s)
 - *weight*: weight for the new objective(s); **negative values invert the sense of the objective**
 - *abstol*: absolute tolerance for the new objective(s)
 - *reltol*: relative tolerance for the new objective(s)



Note: *A multi-objective problem may only contain linear objective functions. The problem itself may be of any kind supported by Xpress, but nonlinear objective terms must be modeled using a transfer variable*

Optimizing with multiple objectives

- Approaches followed by the Optimizer for solving multi-objective problems:
 - Blended (or Archimedian) approach:
 - Applied when objectives have equal priority (their weights may be equal or different)
 - Weighted sum optimization, setting as objective function the linear combination of the added objectives and their weights (weights default to 1 if left undefined, giving an equally-weighted blend)

Optimizing with multiple objectives

- Approaches followed by the Optimizer for solving multi-objective problems:
 - **Blended (or Archimedian) approach:**
 - Applied when **objectives have equal priority** (their weights may be equal or different)
 - Weighted sum optimization, setting as objective function the linear combination of the added objectives and their weights (weights default to 1 if left undefined, giving an equally-weighted blend)
 - **Lexicographic (or preemptive) approach:**
 - Applied when **each objective has a different priority and a unit weight**
 - Xpress will solve the problem once for each distinct objective priority that is defined
 - All objectives from previous iterations are fixed to their optimal values within the tolerances specified in the configuration of the individual objective:

```
objective <= optimal_value * (1 + reltol) + abstol  # for minimization obj.  
objective >= optimal_value * (1 - reltol) - abstol  # for maximization obj.
```


Optimizing with multiple objectives

- Approaches followed by the Optimizer for solving multi-objective problems:
 - **Blended (or Archimedian) approach:**
 - Applied when **objectives have equal priority** (their weights may be equal or different)
 - Weighted sum optimization, setting as objective function the linear combination of the added objectives and their weights (weights default to 1 if left undefined, giving an equally-weighted blend)
 - **Lexicographic (or preemptive) approach:**
 - Applied when **each objective has a different priority and a unit weight**
 - Xpress will solve the problem once for each distinct objective priority that is defined
 - All objectives from previous iterations are fixed to their optimal values within the tolerances specified in the configuration of the individual objective:

```
objective <= optimal_value * (1 + reltol) + abstol    # for minimization obj.  
objective >= optimal_value * (1 - reltol) - abstol    # for maximization obj.
```
 - **Hybrid approach:**
 - Applied when **objectives have both different priorities and different weights**
 - Xpress will solve the problem once for each distinct objective priority defined, optimizing in each iteration a linear combination of the objective functions with the same priority

Optimizing with multiple objectives

- Examples:

```
# blended (weighted sum) approach: equal priority, different weights
p.addObjective(2*x + y, priority=0, weight=-0.7) # maximize, higher weight
p.addObjective(y, priority=0, weight=0.3)        # minimize, lower weight

# lexicographic approach with setObjective()
p.setObjective(xp.Dot(x, return), sense=xp.ObjSense.MAXIMIZE, priority=1) # max. return
p.addObjective(variance, priority=0, weight=-1)                          # minimize risk

# hybrid approach with three objectives
p.addObjective(xp.Sum(x), priority=1, weight=0.5, reltol=0.1)
p.addObjective(xp.Dot(A,x), priority=1, weight=0.3)
p.addObjective(xp.Dot(B,x), priority=0, weight=-0.2)
```



Hint: Check the *MULTIOBJOPS control* to configure the behaviour of the optimizer when solving multi-objective problems

Python Notebook [PN-6]: Optimizing with multiple objectives

- Navigate to [Exercise 6 - Multi-objective optimization](#) in the notebook
- 1. Blended (or Archimedean) and Lexicographic approaches:
 - Analyze the code in the first code cell, run it, and visualize the generated efficient frontier
 - Read the second code cell and run it. Check the solver logs and confirm that:
 - The Optimizer performs two solving procedures, printing the solver log for each one sequentially
 - The value for each objective is printed in the end of all runs
 - The resulting solution falls within the generated efficient frontier
- 2. *Optional: Goal programming*
 - Read the problem description to become familiar with the model
 - Run the code cell and analyze the log. Which target is not achieved by the final solution ?



www.fico.com/optimization